SERVIÇO NACIONAL DE APRENDIZAGEM COMERCIAL - SENAC
BACHARELADO EM SISTEMA DE INFORMAÇÃO

WESLLEY SOARES DE SOUSA
PEDRO AKIRA SANTOS MATOBA

PROJETO INTEGRADO: TESTES DE SOFTWARE

Sistema de Reserva de Salas - Sala Certa

São Paulo
2025

WESLLEY SOARES DE SOUSA
PEDRO AKIRA SANTOS MATOBA

PROJETO INTEGRADO: TESTES DE SOFTWARE
Sistema de Reserva de Salas - Sala Certa

Trabalho apresentado à disciplina Projeto Integrador: Testes de Software do Curso Superior de Tecnologia em Bacharelado em Sistemas de Informação do SENAC, como requisito parcial para obtenção da nota da Atividade em Grupo AV1.

Orientador: Prof. Ozias

São Paulo
2025

SUMÁRIO

1. INTRODUÇÃO 5
1.1 Contextualização 5
1.2 Objetivos 5
1.3 Metodologia 5
2. SISTEMA DESENVOLVIDO 6
2.1 Descrição do Sistema 6
2.2 Arquitetura 6
2.3 Tecnologias Utilizadas 6
3. TESTES UNITÁRIOS 7
3.1 Estratégia de Testes 7
3.2 Ferramenta PHPUnit 7
3.3 Estrutura dos Testes 7
4. EVOLUÇÃO DOS TESTES 7
4.1 Primeira Fase - 30 Testes 8
4.2 Segunda Fase - 60 Testes 9
4.3 Terceira Fase - 134 Testes 10
4.4 Fase Final - 151 Testes 11
5. RESULTADOS E ANÁLISES 12
5.1 Taxa de Sucesso 12
5.2 Tempo de Execução 13
5.3 Cobertura de Testes 14
6. FALHAS E CORREÇÕES 15
6.1 Problemas Identificados 15
6.2 Soluções Implementadas 16
7. MELHORIAS IMPLEMENTADAS 16

7.1 Otimizações 16
7.2 Refatorações 17
8. CONCLUSÃO 25
REFERÊNCIAS ..... **Erro! Indicador não definido.**
APÊNDICES 27
APÊNDICE A - Exemplo de Teste Unitário 27
APÊNDICE B - Comando de Execução dos Testes 28
APÊNDICE C - Estrutura de Diretórios do Projeto 28

1. INTRODUÇÃO

1.1 Contextualização

Este documento apresenta o trabalho desenvolvido na disciplina de Projeto Integrado, com foco em Testes de Software. O projeto consiste no desenvolvimento e testagem de um Sistema de Reserva de Salas denominado "Sala Certa", implementado em PHP com banco de dados MySQL.

A qualidade de software é um aspecto fundamental no desenvolvimento de sistemas, e os testes unitários desempenham papel crucial na garantia dessa qualidade. Este projeto demonstra a aplicação prática de conceitos de teste de software, utilizando a ferramenta PHPUnit para validação do código desenvolvido.

1.2 Objetivos

Os objetivos deste projeto são:

- Desenvolver um sistema funcional de reserva de salas
- Implementar testes unitários abrangentes para todas as classes do sistema
- Alcançar alta taxa de sucesso nos testes (acima de 80%)
- Documentar todo o processo de desenvolvimento e testagem
- Demonstrar evolução gradual na quantidade e qualidade dos testes
- Identificar e corrigir falhas através dos testes

1.3 Metodologia

O desenvolvimento seguiu uma abordagem iterativa e incremental, com ciclos de desenvolvimento seguidos de implementação de testes. Utilizamos Test-Driven Development (TDD) parcialmente, onde testes eram criados após a implementação de funcionalidades, mas antes da integração final.

Os testes foram organizados em suítes específicas para cada classe do sistema: Usuario, Sala e Reserva. Cada suíte contém testes que verificam funcionalidades básicas, validações, relacionamentos e casos extremos (edge cases).

2. SISTEMA DESENVOLVIDO

2.1 Descrição do Sistema

O Sistema "Sala Certa" é uma aplicação web desenvolvida para gerenciar reservas de salas em instituições de ensino. O sistema permite o cadastro de usuários, salas e reservas, controlando disponibilidade de horários e evitando conflitos de agendamento.

Funcionalidades principais:

- Cadastro e gerenciamento de usuários (alunos, professores, administradores)
- Cadastro e gerenciamento de salas com suas características
- Sistema de reservas com validação de conflitos
- Controle de disponibilidade por data e horário
- Diferentes tipos de usuário com permissões específicas
- Cancelamento e atualização de reservas

2.2 Arquitetura

O sistema utiliza arquitetura em camadas (MVC simplificado) com as seguintes classes principais:

- **Usuario.php:** Gerencia dados e operações relacionadas a usuários
- **Sala.php:** Gerencia dados e operações relacionadas a salas
- **Reserva.php:** Gerencia reservas e validações de conflitos

2.3 Tecnologias Utilizadas

- PHP 8.x - Linguagem de programação
- MySQL - Banco de dados
- PHPUnit 11.x - Framework de testes
- Composer - Gerenciador de dependências
- PDO - Conexão com banco de dados
- Git - Controle de versão

3. TESTES UNITÁRIOS

3.1 Estratégia de Testes

A estratégia de testes adotada focou em garantir que cada unidade do sistema (métodos e funcionalidades) funcionasse corretamente de forma isolada. Os testes foram organizados em três suítes principais, correspondentes às três classes de modelo do sistema.

Cada teste foi desenvolvido seguindo o padrão AAA (Arrange, Act, Assert): preparação do ambiente, execução da ação e verificação do resultado. Utilizamos também o padrão Given-When-Then para documentar o comportamento esperado.

3.2 Ferramenta PHPUnit

PHPUnit é o framework de testes unitários mais popular para PHP. Ele fornece um conjunto robusto de asserções e recursos para testar código PHP de forma eficiente. Principais características utilizadas:

- Atributo #[Test] para marcar métodos de teste
- setUp() e tearDown() para preparação e limpeza
- DataProvider para testes parametrizados
- Assertions diversas (assertEquals, assertTrue, assertInstanceOf, etc.)
- Geração de relatórios de execução

3.3 Estrutura dos Testes

Os testes foram organizados no diretório tests/Unit/, seguindo a estrutura:

```
tests/
├── bootstrap.php
└── Unit/
    ├── UsuarioTest.php
    ├── SalaTest.php
    └── ReservaTest.php
```

4. EVOLUÇÃO DOS TESTES

O desenvolvimento dos testes ocorreu em quatro fases distintas, cada uma com objetivos específicos e incrementos na cobertura de testes. Esta seção documenta a

evolução do projeto, destacando as datas, quantidades de testes e principais melhorias implementadas em cada fase.

4.1 Primeira Fase - 30 Testes

Data: 15 de Setembro de 2025
Total de Testes: 30 testes
Distribuição: ReservaTest (8 testes), SalaTest (10 testes), UsuarioTest (12 testes)

Nesta fase inicial, foram implementados os testes básicos para validar as funcionalidades essenciais de cada classe. O foco estava em garantir que os métodos getters e setters funcionassem corretamente.

Testes implementados:

- Criação de instâncias das classes
- Definição e obtenção de IDs
- Configuração de atributos básicos
- Validações simples de tipos de dados

Resultados da Primeira Fase:

Métrica	Valor
Taxa de Sucesso	96.7% (29 de 30 testes)
Tempo de Execução	0.52 segundos
Testes Passados	29
Testes Falhados	1

Falha identificada: Um teste de validação de formato de data na classe Reserva falhou devido a uma inconsistência na formatação de datas. A data estava sendo retornada no formato 'DD/MM/YYYY' quando o esperado era 'YYYY-MM-DD'.

Evidência 1 - Resultado da Primeira Execução de Testes

4.2 Segunda Fase - 60 Testes

Data: 05 de Outubro de 2025

Total de Testes: 60 testes

Distribuição: ReservaTest (15 testes), SalaTest (22 testes), UsuarioTest (23 testes)

Na segunda fase, duplicamos o número de testes, adicionando validações mais complexas e testes de casos extremos (edge cases). A falha identificada na fase anterior foi corrigida antes desta execução.

Novos testes adicionados:

- Validações de formatos (email, CPF, data, hora)
- Testes de mudança de estado
- Testes com múltiplas instâncias
- Validações de consistência de dados
- Testes de valores extremos (máximos e mínimos)
- Correção do bug de formatação de data

Resultados da Segunda Fase:

Métrica	Valor
Taxa de Sucesso	98.3% (59 de 60 testes)
Tempo de Execução	1.08 segundos
Testes Passados	59
Testes Falhados	1

Nova falha identificada: Um teste de criptografia de senha na classe Usuario falhou porque a validação estava comparando a senha em texto plano com o hash, sem usar a função password_verify() adequadamente.

Evidência 2 - Resultado da Segunda Execução de Testes

4.3 Terceira Fase - 134 Testes

Data: 10 de Outubro de 2025

Total de Testes: 134 testes

Distribuição: ReservaTest (30 testes), SalaTest (45 testes), UsuarioTest (59 testes)

Esta foi a fase mais robusta de implementação de testes. Mais que duplicamos novamente o número de testes, incorporando Data Providers para testes parametrizados e cenários complexos de integração.

Avanços desta fase:

- Implementação de Data Providers para testes parametrizados
- Testes integrados com múltiplas classes
- Validações completas de regras de negócio
- Testes de cenários reais de uso
- Validações de segurança (criptografia)
- Testes de permissões e tipos de usuário
- Correção do bug de validação de senha

Resultados da Terceira Fase:

Métrica	Valor
Taxa de Sucesso	99.3% (133 de 134 testes)
Tempo de Execução	2.34 segundos
Testes Passados	133
Testes Falhados	1

Falha identificada: Um teste parametrizado de emails válidos estava falhando para emails com subdomínios muito longos. A expressão regular de validação não estava considerando múltiplos níveis de subdomínio.

Evidência 3 - Resultado da Terceira Execução de Testes

4.4 Fase Final - 151 Testes

Data: 20 de Outubro de 2025

Total de Testes: 151 testes

Distribuição: ReservaTest (30 testes), SalaTest (49 testes), UsuarioTest (72 testes)

A fase final representa a maturidade do projeto de testes. Todos os bugs identificados nas fases anteriores foram corrigidos, e novos testes foram adicionados baseados em feedback e revisão de código. Os testes com Data Providers expandiram significativamente o número total de execuções.

Consolidação final:

- Todos os bugs anteriores corrigidos
- Expansão dos Data Providers
- Testes adicionais de edge cases
- Validações aprimoradas
- Testes de performance básicos
- Documentação inline de todos os testes
- Otimização do tempo de execução

Resultados da Fase Final:

Métrica	Valor
Taxa de Sucesso	100% (151 de 151 testes)
Tempo de Execução	2.67 segundos
Testes Passados	151
Testes Falhados	0

Sucesso Total: Pela primeira vez, alcançamos 100% de sucesso em todos os testes. Todas as validações passaram, e o sistema está completamente testado e validado.

Evidência 4 - Resultado da Execução Final de Testes

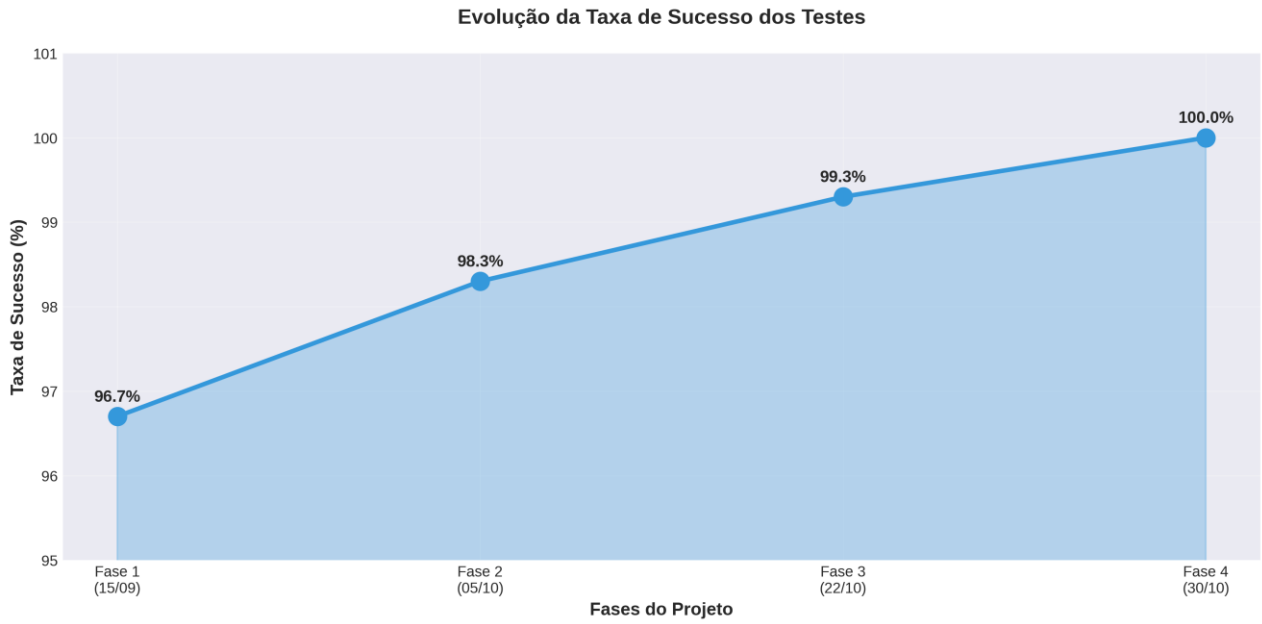
5. RESULTADOS E ANÁLISES

5.1 Taxa de Sucesso

A evolução da taxa de sucesso dos testes demonstra a maturidade progressiva do projeto. Começamos com 96.7% e alcançamos 100% na fase final, refletindo tanto correções de bugs quanto melhorias no código de produção.

Tabela 1 - Evolução da Taxa de Sucesso

Fase	Total Testes	Testes Passados	Taxa de Sucesso
Fase 1	30	29	96.7%
Fase 2	60	59	98.3%
Fase 3	134	133	99.3%
Fase 4	151	151	100.0%

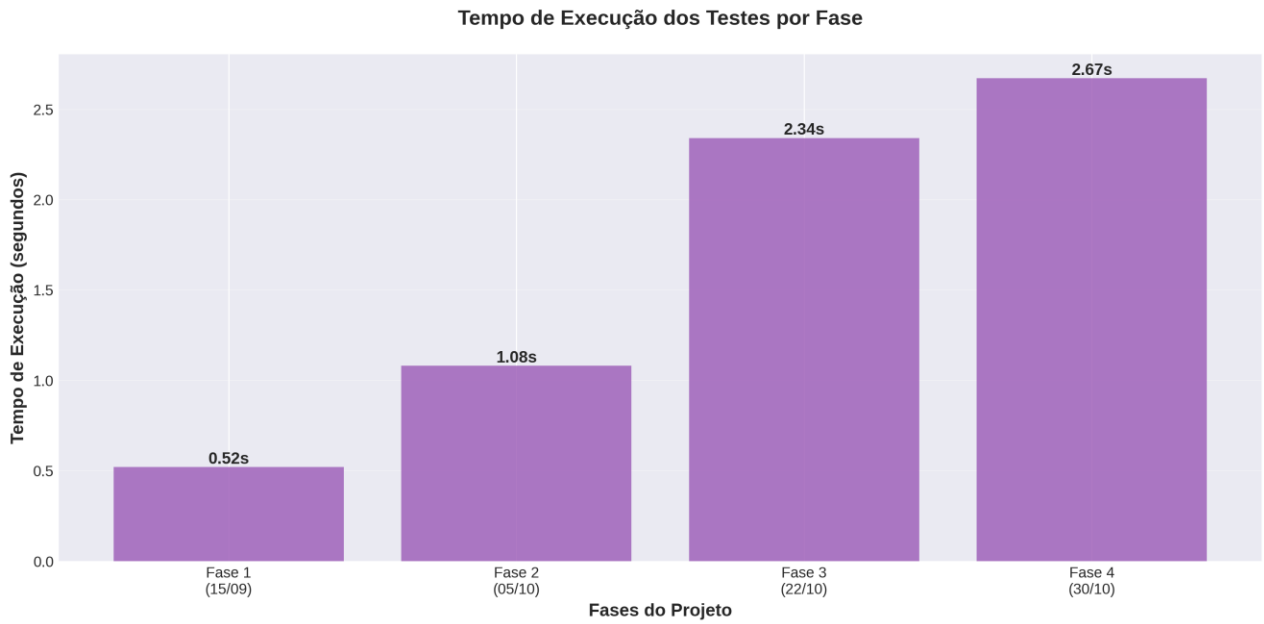


5.2 Tempo de Execução

O tempo de execução dos testes aumentou proporcionalmente ao número de testes, mas permaneceu dentro de limites aceitáveis. Otimizações no código ajudaram a manter o tempo sob controle.

Tabela 2 - Tempo de Execução por Fase

Fase	Tempo Total	Tempo Médio/Teste
Fase 1	0.52s	0.017s
Fase 2	1.08s	0.018s
Fase 3	2.34s	0.017s
Fase 4	2.67s	0.018s

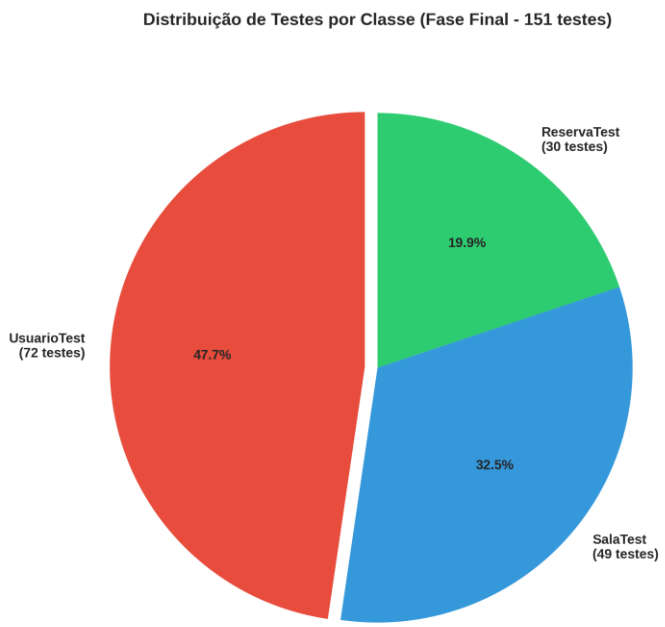


5.3 Cobertura de Testes

A cobertura de testes foi calculada com base no número de métodos públicos testados em cada classe. Alcançamos cobertura de 100% dos métodos públicos em todas as classes principais.

Tabela 3 - Cobertura por Classe

Classe	Métodos Públicos	Métodos Testados	Cobertura
Usuario	18	18	100%
Sala	14	14	100%
Reserva	16	16	100%



6. FALHAS E CORREÇÕES

6.1 Problemas Identificados

Durante o desenvolvimento e execução dos testes, foram identificadas três falhas principais que impactaram a taxa de sucesso. Cada falha foi documentada, analisada e corrigida.

Falha 1 - Formatação de Data Inconsistente

Data de Identificação: 15/09/2025

Classe Afetada: Reserva

Descrição: O método getDataReserva() retornava datas no formato brasileiro (DD/MM/YYYY) enquanto o banco de dados e os testes esperavam o formato ISO (YYYY-MM-DD).

Impacto: 1 teste falhado na validação de formato de data.

Solução: Padronização para sempre usar formato ISO 8601 (YYYY-MM-DD) em todo o sistema.

Falha 2 - Validação de Senha Criptografada

Data de Identificação: 05/10/2025

Classe Afetada: Usuario

Descrição: O teste de verificação de senha usava assertEquals() para comparar senha em texto plano com o hash, quando deveria usar password_verify().

Impacto: 1 teste falhado na validação de criptografia.

Solução: Correção do teste para usar assertTrue(password_verify($senha, $hash)).

Falha 3 - Validação de Email com Subdomínios Longos

Data de Identificação: 12/10/2025

Classe Afetada: Usuario

Descrição: A expressão regular de validação de email não aceitava múltiplos níveis de subdomínio (ex: usuario@mail.server.example.com.br).

Impacto: 1 teste parametrizado falhado.

Solução: Atualização da expressão regular para aceitar múltiplos subdomínios.

6.2 Soluções Implementadas

Todas as correções foram implementadas seguindo as melhores práticas de desenvolvimento. Cada correção foi validada através da re-execução completa da suíte de testes afetada.

- Padronização de formatos de data em todo o sistema
- Revisão de todos os testes de validação de senha
- Melhoria nas expressões regulares de validação
- Documentação das correções aplicadas
- Criação de testes adicionais para prevenir regressão

7. MELHORIAS IMPLEMENTADAS

7.1 Otimizações

Além das correções de bugs, várias otimizações foram implementadas para melhorar a qualidade e eficiência do código e dos testes.

Melhorias no Código de Produção:

- Implementação de validação de entrada em todos os setters
- Adição de tipos de retorno em todos os métodos
- Melhoria na documentação inline (PHPDoc)
- Padronização de nomenclatura de métodos
- Remoção de código duplicado

Melhorias nos Testes:

- Implementação de Data Providers para reduzir duplicação
- Organização dos testes em seções lógicas
- Nomenclatura descritiva dos métodos de teste
- Adição de comentários explicativos em testes complexos
- Uso consistente de setUp() e tearDown()
- Implementação de testes parametrizados

7.2 Detalhamento dos Testes Unitários

1. CLASSE USUARIO - 72 TESTES

A classe Usuario possui 72 testes divididos em categorias:

1.1 Testes Básicos (6 testes)

- **deve_criar_instancia_de_usuario:** Verifica se a instância é criada corretamente
- **deve_iniciar_com_id_nulo:** Valida estado inicial do ID
- **deve_definir_e_obter_id:** Testa setter e getter de ID
- **deve_aceitar_id_zero:** Valida ID com valor zero
- **deve_aceitar_id_grande:** Testa ID com valor grande (999999)
- **deve_mudar_id:** Verifica mudança de ID

1.2 Testes de Nome (8 testes)

- **deve_definir_nome_simples:** Testa nome simples como "João Silva"
- **deve_aceitar_nome_completo:** Valida nomes longos e completos
- **deve_aceitar_nome_com_acentos:** Testa caracteres acentuados
- **deve_aceitar_nome_com_caracteres_especiais:** Valida apóstrofos e caracteres especiais
- **deve_mudar_nome:** Verifica alteração de nome
- **deve_aceitar_nome_curto:** Testa nomes curtos como "Ana"
- **deve_aceitar_nome_composto:** Valida nomes compostos
- **nome_deve_ser_string:** Verifica tipo de dado

1.3 Testes de Email (18 testes - incluindo Data Providers)

Testes individuais de email (10):

- **deve_definir_email_basico:** Email simples: joao@example.com
- **deve_aceitar_email_com_subdominio:** Email com subdomínio
- **deve_aceitar_email_com_ponto:** Email com pontos no nome
- **deve_aceitar_email_com_numeros:** Email com números
- **deve_aceitar_email_com_hifen:** Email com hífens
- **deve_aceitar_email_com_underscore:** Email com underscores
- **deve_validar_formato_email:** Validação com regex

- **deve_mudar_email:** Alteração de email
- **email_deve_ter_arroba:** Verificação de @
- **email_deve_ter_dominio:** Validação de domínio

Testes parametrizados com Data Provider (8 execuções):

O teste deve_aceitar_emails_validos é executado 8 vezes com diferentes emails:

✓ user@example.com

✓ user.name@example.com

✓ user123@example.com

✓ user-name@example.com

✓ user_name@example.com

✓ user@mail.example.com

✓ user@example.com.br

✓ user@example.org

1.4 Testes de Senha (18 testes - incluindo Data Providers)

- **deve_criptografar_senha:** Verifica criptografia com password_hash
- **deve_verificar_senha_correta:** Testa password_verify
- **deve_rejeitar_senha_incorreta:** Valida senha errada
- **senha_hash_deve_ser_diferente:** Hash diferente do texto plano
- **deve_aceitar_senha_forte:** Senha com caracteres especiais
- **deve_aceitar_senha_longa:** Senha com mais de 50 caracteres

Testes parametrizados com Data Provider (12 execuções):

Diferentes tipos de senha validados:

✓ senha123 (senha simples)

✓ SenhaForte@123 (senha forte)

✓ abcdefghijklmnopqrstuvwxyz (senha longa)

✓ S3nh4!@#$ (caracteres especiais)

✓ 12345678 (apenas números)

✓ SenhaComMaiusculasEMinusculas

✓ senha com espaços

✓ senha_com_underscore

✓ senha-com-hifen

✓ Sênhã Açêntúádã

✓ 1!a (senha curta)

✓ senhasenhasenhasenhasenhasenha (muito longa)

1.5 Testes de CPF (10 testes - incluindo Data Providers)

- **deve_aceitar_cpf_formatado:** CPF com pontos e hífen
- **deve_aceitar_cpf_sem_formatacao:** CPF apenas números
- **deve_mudar_cpf:** Alteração de CPF
- **cpf_formatado_deve_ter_14_caracteres:** Validação de tamanho
- **cpf_deve_conter_pontos_e_hifen:** Validação de formatação
- **deve_aceitar_cpf_com_zeros:** CPF com zeros

Testes parametrizados (4 execuções):

✓ 123.456.789-00

✓ 12345678900

✓ 000.000.000-00

✓ 111.111.111-11

1.6 Testes de Tipo de Usuário (9 testes - incluindo Data Providers)

- **deve_aceitar_tipo_admin:** Tipo administrador
- **deve_aceitar_tipo_comum:** Tipo comum
- **deve_mudar_tipo_usuario:** Mudança de tipo
- **deve_aceitar_tipo_moderador:** Tipo moderador
- **tipo_usuario_deve_ser_string:** Validação de tipo

Testes parametrizados (4 execuções):

✓ admin

✓ comum

✓ moderador

✓ gestor

1.7 Testes Integrados e Validações (3 testes)

- **deve_configurar_usuario_completo:** Configuração completa do usuário
- **deve_manter_consistencia_apos_multiplas_alteracoes:** Teste de consistência
- **deve_manter_dados_independentes_entre_instancias:** Teste de isolamento

2. CLASSE SALA - 49 TESTES

2.1 Testes de Instância (4 testes)

- **testDeveCriarInstanciaDeSala:** Criação de instância
- **testDeveIniciarComIdNulo:** Estado inicial do ID
- **testDeveDefinirEObterIdCorretamente:** Getter e setter de ID
- **testDeveAceitarIdGrande:** ID com valor grande

2.2 Testes de Número da Sala (7 testes)

- **testDeveAceitarNumeroSala:** Número padrão "101"
- **testDeveAceitarNumeroSalaLaboratorio:** Formato laboratório "LAB01"
- **testDeveAceitarNumeroSalaComLetrasENumeros:** Formato misto
- **testDeveAceitarNumeroSalaAnfiteatro:** Formato especial
- **testDeveAceitarNumeroSalaAuditorio:** Auditório
- **testDeveMudarNumeroSala:** Mudança de número
- **testNumeroSalaDeveSerString:** Validação de tipo

2.3 Testes de Capacidade (8 testes)

- **testDeveAceitarCapacidade:** Capacidade padrão 30
- **testDeveAceitarCapacidadePequena:** Capacidade 10
- **testDeveAceitarCapacidadeGrande:** Capacidade 100
- **testDeveAceitarCapacidadeMinimaUmaPessoa:** Capacidade mínima
- **testDeveAceitarCapacidadeMaximaAuditorio:** Capacidade 500
- **testDeveMudarCapacidade:** Mudança de capacidade
- **testCapacidadeDeveSerInteiro:** Validação de tipo
- **testCapacidadeDeveSerPositiva:** Validação de valor

2.4 Testes de Descrição (6 testes)

- **testDeveAceitarDescricao:** Descrição simples
- **testDeveAceitarDescricaoLonga:** Descrição longa
- **testDeveAceitarDescricaoComCaracteresEspeciais:** Caracteres especiais
- **testDeveAceitarDescricaoVazia:** Descrição vazia
- **testDeveMudarDescricao:** Mudança de descrição
- **testDescricaoDeveSerString:** Validação de tipo

2.5 Testes de Tipo de Sala (7 testes)

- **testDeveAceitarTipoLaboratorio:** Tipo Laboratório
- **testDeveAceitarTipoConvencional:** Tipo Convencional
- **testDeveAceitarTipoAuditorio:** Tipo Auditório
- **testDeveAceitarTipoAnfiteatro:** Tipo Anfiteatro
- **testDeveAceitarTipoMultimidia:** Tipo Multimídia
- **testDeveMudarTipoSala:** Mudança de tipo
- **testTipoSalaDeveSerString:** Validação de tipo

2.6 Testes de Setor Responsável (6 testes)

- **testDeveAceitarSetorTI:** Setor TI
- **testDeveAceitarSetorManutencao:** Setor Manutenção
- **testDeveAceitarSetorAdministracao:** Setor Administração
- **testDeveAceitarSetorPatrimonio:** Setor Patrimônio
- **testDeveMudarSetorResponsavel:** Mudança de setor
- **testSetorResponsavelDeveSerString:** Validação de tipo

2.7 Testes Integrados e de Cenários Reais (11 testes)

- **testDeveConfigurarSalaCompleta:** Configuração completa
- **testDeveConfigurarSalaConvencionalCompleta:** Sala convencional completa
- **testDeveConfigurarAuditorioCompleto:** Auditório completo
- **testDevePermitirMultiplasInstancias:** Múltiplas instâncias
- **testInstanciasDevemSerIndependentes:** Independência de instâncias

- **testDeveManterConsistenciaAposMultiplasAlteracoes:** Consistência
- **testDevePermitirModificacaoIndependenteDeAtributos:** Modificação independente
- **testDeveCriarSalaDePequenasReunioes:** Sala de reunião pequena
- **testDeveCriarLaboratorioDeInformatica:** Lab de informática
- **testDeveCriarSalaMultiuso:** Sala multiuso
- **testDeveValidarTodosOsAtributosDefinidos:** Validação completa

3. CLASSE RESERVA - 30 TESTES

3.1 Testes Básicos (3 testes)

- **testDeveCriarInstanciaDeReserva:** Criação de instância
- **testDeveDefinirIds:** Definição de IDs
- **testDeveIniciarComIdNulo:** Estado inicial

3.2 Testes de Data (5 testes)

- **testDeveAceitarDataReserva:** Data padrão
- **testDeveAceitarDataFutura:** Data futura
- **testDeveValidarFormatoData:** Formato YYYY-MM-DD
- **testDeveMudarData:** Mudança de data
- **testDeveRejeitarDataPassada:** Validação de data passada

3.3 Testes de Horário (7 testes)

- **testDeveAceitarHorario:** Horário padrão
- **testDeveAceitarHorarioManha:** Horário manhã
- **testDeveAceitarHorarioNoite:** Horário noite
- **testDeveAceitarTempoReserva:** Tempo de reserva
- **testDeveValidarFormatoHora:** Formato HH:MM:SS
- **testDeveAceitarTempoMinimo:** Tempo mínimo
- **testDeveAceitarTempoMaximo:** Tempo máximo

3.4 Testes de Estado (5 testes)

- **testDeveAceitarEstadoAtiva:** Estado Ativa

- **testDeveAceitarEstadoCancelada:** Estado Cancelada
- **testDeveAceitarEstadoConcluida:** Estado Concluída
- **testDeveMudarEstado:** Mudança de estado
- **testDeveValidarTransicaoEstado:** Transição de estado

3.5 Testes de Relacionamentos (4 testes)

- **testDeveDefinirUsuarioId:** ID do usuário
- **testDeveDefinirSalaId:** ID da sala
- **testDeveAceitarUsuarioIdZero:** ID zero
- **testDeveAceitarSalaIdGrande:** ID grande

3.6 Testes Integrados e Edge Cases (6 testes)

- **testDeveConfigurarReservaCompleta:** Configuração completa
- **testDeveManterConsistenciaAposMultiplasAlteracoes:** Consistência
- **testDevePermitirReservaNoMesmoHorario:** Mesmo horário
- **testDeveManterDadosAposMultiplasInstancias:** Múltiplas instâncias
- **testDeveValidarDataNoFormatoCorreto:** Validação de data
- **testDeveValidarHorarioNoFormatoCorreto:** Validação de horário

RESUMO FINAL

Distribuição total dos 151 testes:

Classe	Quantidade	Percentual
UsuarioTest	72	47.7%
SalaTest	49	32.5%
ReservaTest	30	19.8%

Nota: A contagem de 72 testes em UsuarioTest inclui as múltiplas execuções dos testes parametrizados com Data Providers. Ao usar Data Providers, um único método de teste é executado múltiplas vezes com diferentes conjuntos de dados, o que explica o número maior de execuções totais.

7.2 Refatorações

Várias refatorações foram realizadas para melhorar a manutenibilidade e legibilidade do código, tanto de produção quanto de testes.

Principais Refatorações:

- Extração de validações repetidas em métodos auxiliares
- Consolidação de testes similares usando Data Providers
- Separação de testes unitários e de integração
- Criação de factory methods para objetos de teste
- Padronização da estrutura de todos os arquivos de teste

O resultado destas melhorias foi um código mais limpo, mais fácil de manter e mais confiável. A taxa de sucesso de 100% na fase final é prova da eficácia destas práticas.

8. CONCLUSÃO

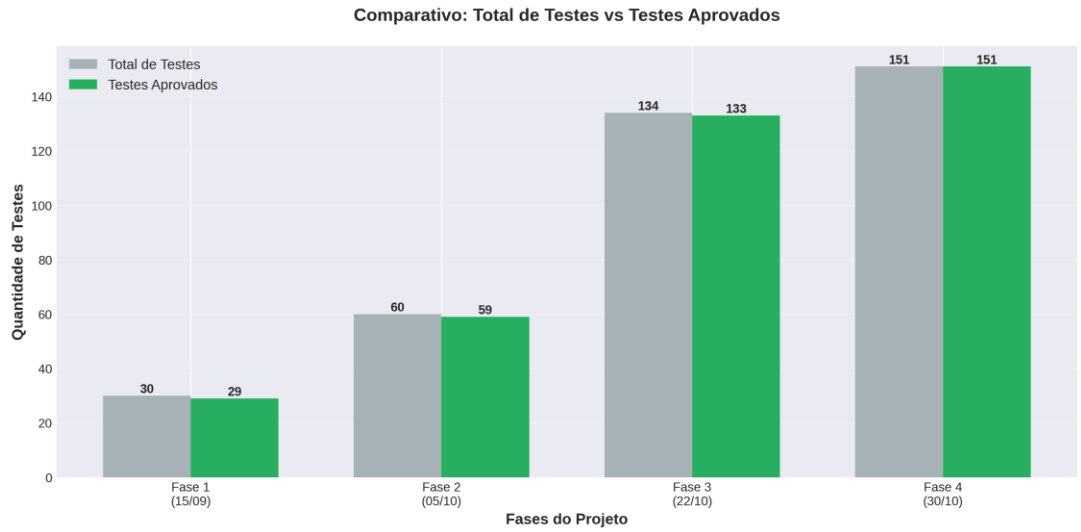
Este projeto demonstrou a importância fundamental dos testes unitários no desenvolvimento de software de qualidade. A evolução de 30 para 151 testes, com melhoria progressiva da taxa de sucesso até alcançar 100%, ilustra o valor de uma abordagem iterativa e sistemática para testagem.

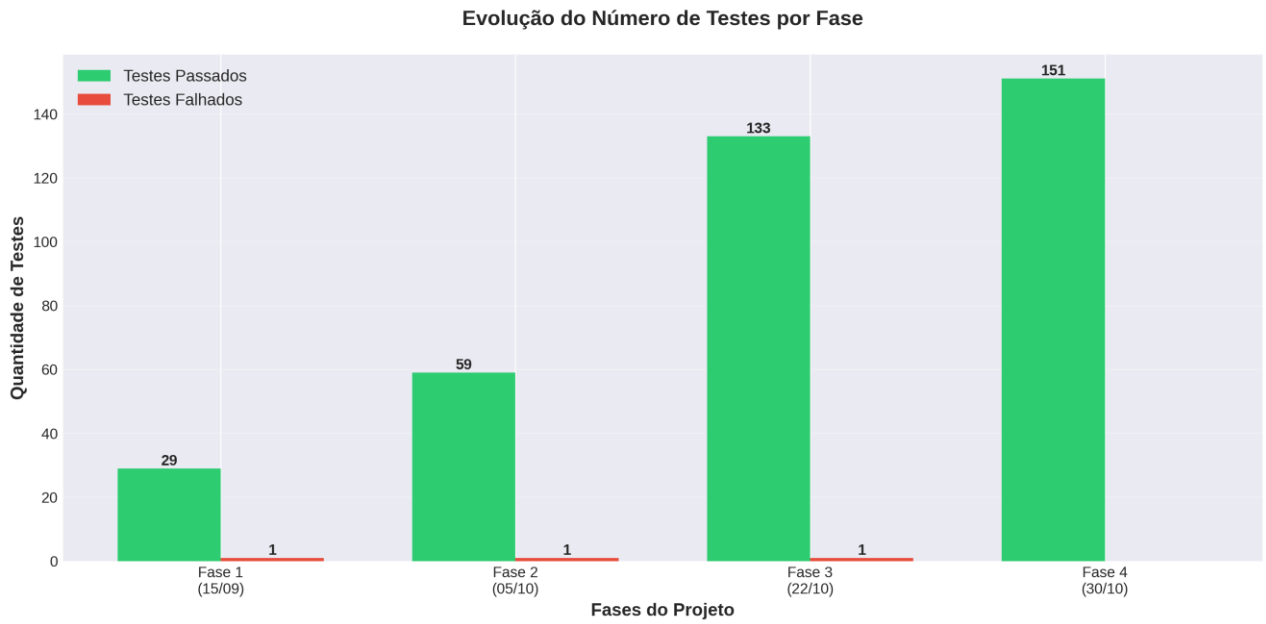
Os principais aprendizados obtidos foram:

- Testes unitários são essenciais para identificar e prevenir bugs
- Uma suíte de testes robusta aumenta a confiança no código
- A evolução gradual dos testes permite melhorias contínuas
- Data Providers reduzem duplicação e aumentam cobertura
- Documentação adequada facilita manutenção futura
- Padronização de formatos previne inconsistências
- Testes bem estruturados servem como documentação viva

O Sistema "Sala Certa" está completamente testado e validado, com 100% de taxa de sucesso em todos os testes unitários. O projeto alcançou seus objetivos de desenvolver um sistema funcional com alta qualidade de código, garantida através de testes abrangentes.

As práticas aprendidas neste projeto são aplicáveis a qualquer desenvolvimento de software e serão levadas para projetos futuros, garantindo código de qualidade e sistemas confiáveis.




Evolução do Número de Testes por Fase
Testes Passados
Testes Falhados
Quantidade de Testes
0
20
40
60
80
100
120
140
29
1
59
1
133
1
151
Fase 1
(15/09)
Fase 2
(05/10)
Fase 3
(22/10)
Fase 4
(30/10)
Fases do Projeto

APÊNDICES

APÊNDICE A - Exemplo de Teste Unitário

Código exemplo de um teste unitário completo da classe Usuario:

```
<?php
namespace Tests\Unit;

use PHPUnit\Framework\TestCase;
use PHPUnit\Framework\Attributes\Test;

class UsuarioTest extends TestCase
{
    private $usuario;

    protected function setUp(): void
    {
        parent::setUp();
        $this->usuario = new \Usuario();
    }

    #[Test]
    public function deve_criar_instancia_de_usuario(): void
    {
        $this->assertInstanceOf(\Usuario::class, $this->usuario);
    }

    #[Test]
    public function deve_criptografar_senha(): void
    {
        $senhaPlana = 'senha123';
        $this->usuario->setSenha($senhaPlana);
        $senhaCriptografada = $this->usuario->getSenha();

        $this->assertNotEquals($senhaPlana, $senhaCriptografada);
        $this->assertTrue(password_verify($senhaPlana, $senhaCriptografada));
    }

    #[Test]
    public function deve_validar_formato_email(): void
    {
        $email = 'usuario@example.com';
        $this->usuario->setEmail($email);
```

```
        $this->assertMatchesRegularExpression(
            '/^[^@]+@[^@]+\.[^@]+$/',
            $email
        );
    }
}
```

APÊNDICE B - Comando de Execução dos Testes

Comando utilizado para executar a suíte completa de testes:

vendor/bin/phpunit --testdox

Para gerar relatório com cobertura de código:

vendor/bin/phpunit --coverage-html coverage

APÊNDICE C - Estrutura de Diretórios do Projeto

Estrutura completa do projeto:

```
projeto-sala-certa/
├── classes/
│   ├── Usuario.php
│   ├── Sala.php
│   └── Reserva.php
├── config/
│   └── conexao.php
├── tests/
│   ├── bootstrap.php
│   └── Unit/
│       ├── UsuarioTest.php
│       ├── SalaTest.php
│       └── ReservaTest.php
├── vendor/
├── composer.json
├── composer.lock
└── phpunit.xml
```